

Security and Privacy Analyses of Internet of Things Children's Toys

Gordon Chu, Noah Apthorpe¹, and Nick Feamster

Abstract—This paper investigates the security and privacy of Internet-connected children's smart toys through case studies of three commercially available products. We conduct network and application vulnerability analyses of each toy using static and dynamic analysis techniques, including application binary decompilation and network monitoring. We discover several publicly undisclosed vulnerabilities that violate the Children's Online Privacy Protection Rule as well as the toys' individual privacy policies. These vulnerabilities, especially security flaws in network communications with first-party servers, are indicative of a disconnect between many Internet of Things toy developers and security and privacy best practices despite increased attention to Internet-connected toy hacking risks.

Index Terms—Data security, Internet of Things (IoT), privacy.

I. INTRODUCTION

INTERNET-CONNECTED children's toys are a subset of Internet of Things (IoT) devices that merit special attention from the security community. If IoT toys are compromised, a cyber predator could communicate with or collect sensitive data about children without being physically near them. The 2015 Hello Barbie hack, in which security researchers were able to remotely access the doll's microphone and record children's conversations [1], clearly demonstrates the importance of securing IoT toys.

Regulators have given special attention to privacy concerns of online services targeted toward children. The Federal Trade Commission's Children's Online Privacy Protection Rule (COPPA) places specific requirements on these services, including that they must "establish and maintain reasonable procedures to protect the confidentiality, security, and integrity of personal information collected from children" [2].

In addition to mandated COPPA compliance, manufacturers of IoT toys provide their own privacy policies to indicate data handling practices and security measures specific to their products. However, consumers have no way to verify whether IoT toys actually follow COPPA regulations or manufacturer privacy policies. Recent high-profile hacks of IoT

devices give reason to doubt the security claims of these products [1], [3]–[5].

In this paper, we analyze three commercially available IoT toys, a hydration tracker, a smart pet, and a fitness band, for application and network security and privacy vulnerabilities. All three products are clearly targeted toward minors on their respective websites and the Google Play store. The mobile application for the hydration tracker toy has 5000+ installs from the Google Play store as of July 2018. The mobile applications for the smart pet and fitness band have 10000+ and 1000+ installs, respectively. For comparison, the Hello Barbie companion app has 5000+ installs. All analyses in this paper were performed with the version of the toys and companion mobile applications available in November 2017.

We chose these specific toys and mobile applications at the request of the state attorney general. We have not disclosed discovered vulnerabilities to the toy vendors due to ongoing attorney general investigations. The attorney general will notify the vendors when appropriate. We keep the identities of the toys anonymous in this paper pending security patches.

We uncover several previously undisclosed vulnerabilities, including a lack of data encryption (HTTP instead of HTTPS), lack of authentication for accessing personally identifiable information (PII), POST token reuse, asymmetric HTTP responses allowing unique ID mining, PII in crash reports to third parties, and secret keys in source code constant files. We then identify how these vulnerabilities constitute violations of both COPPA and the individual toys' privacy policies.

Despite their potentially serious impact, these vulnerabilities are all easily correctable. Their existence indicates substantial developer error, apathy, or ignorance of privacy and security best practices. Even simple and well-known security practices, such as transport-layer encryption, are neglected by developers. These case studies motivate more rigorous auditing of Internet-connected toys by researchers and consumer advocacy groups as well as improved development practices by IoT toy manufacturers.

This paper makes the following contributions.

- 1) Presents previously undisclosed vulnerabilities of three Internet-connected children's toys, informing general observations about the current state of smart toy security and privacy.
 - a) Lack of industry-standard security practices, especially encryption/authentication of communications with first-party cloud services, leaves personal data unprotected, and constitutes violations of

Manuscript received May 9, 2018; revised July 20, 2018; accepted August 15, 2018. Date of publication August 21, 2018; date of current version February 25, 2019. This work was supported in part by the Google Faculty Research Award, in part by the National Science Foundation under Award CNS-1535796 and Award CNS-1539902, and in part by the Princeton University Center for Information Technology Policy Internet of Things Consortium. (Corresponding author: Noah Apthorpe.)

The authors are with the Computer Science Department, Princeton University, Princeton, NJ 08540 USA (e-mail: gordon.chu@princeton.edu; apthorpe@cs.princeton.edu; feamster@cs.princeton.edu).

Digital Object Identifier 10.1109/JIOT.2018.2866423

manufacturer privacy policies and federal COPPA regulation.

- b) Use of common third-party analytics services across smart toys could allow cross-device tracking of child behavior.

II. RELATED WORK

Recent high-profile hacks of smart toys [1], [5], have drawn considerable press attention to the privacy and security risks of Internet-connected devices for children [6], [7]. The academic community has responded to consumer concern with studies auditing smart toys [3], [8]–[10], analyzing children's and parents' interactions with smart toys [11], [12], and proposing technical and legal frameworks or requirements for improving smart toy security and privacy [13]–[17]. This paper fits into the general methodological framework of previous toy security audits in that it systematically explores vulnerabilities in consumer smart toys, provides proof-of-concept exploits, and suggests remediation techniques. However, unlike most previous studies, this paper situates discovered vulnerabilities in the context of federal COPPA regulation and the examined toys' respective privacy policies. By doing so, it provides a glimpse into how IoT toy developers' practices can fail to match their own privacy promises or adhere to federal regulations. These results corroborate the findings of Reyes *et al.* [18] that many Android applications targeted toward children are potentially in violation of COPPA.

III. METHOD

We evaluate the security of three commercially available IoT toys: 1) a hydration tracker; 2) a smart pet; and 3) a fitness band. None of the toys have publicly disclosed vulnerabilities.

We evaluate each toy with static and dynamic analyses to understand its functionality and to discover application and network security vulnerabilities. Finally, we check whether any discovered vulnerabilities violate the toys' privacy policies or COPPA.

Our approach involves commonly used tools for penetration testing and security auditing, emphasizing the breadth and severity of smart toy vulnerabilities that can be discovered using these methods, which are publicly available and have a low barrier to use for potential attackers. The vulnerabilities reported are unlikely comprehensive—future research using more sophisticated techniques, such as taint tracking [19], may uncover additional vulnerabilities; however, we focus on the highest impact issues that are both easily detectable by attackers and easily correctable by manufacturers.

A. Static Analysis

We collected publicly available documentation about each toy, including user manuals and privacy policies. We examined the documentation to understand the capabilities of each toy and to learn which discovered vulnerabilities violate its privacy policy.

We also decompiled the Android mobile application associated with each toy using the *jadx* Dex to Java compiler [20]. We examined the source code for security vulnerabilities,

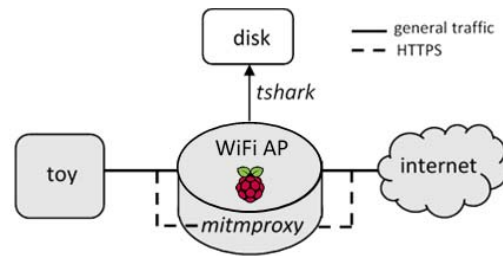


Fig. 1. Packet capture pipeline for recording IoT toy network traffic. All toys were connected to an instrumented WiFi access point implemented on a Raspberry Pi.

such as cleartext passwords, network serialization formats, and metadata about developers.

B. Dynamic Analysis

We collected packet captures from the toys by associating them with an instrumented WiFi access point implemented on a Raspberry Pi (Fig. 1) [21]. HTTPS traffic was decrypted using *mitmproxy* [22]. The command line utility *tshark* [23] was run on the Raspberry Pi to log packets to disk. Recorded packet traces were then analyzed visually with *wireshark* [24] and programmatically with the Python Scapy library [25]. We focused on traffic between the toys and backend cloud servers because it is easy to intercept and subject to the widest range of attacks. Security and privacy evaluations of non-WiFi local communications (Bluetooth, ZigBee, etc.) remain a topic for future work.

We also used the network mapper *nmap* [26] to determine what ports were open on the toys. No port scans were launched against remote servers that the toys communicated with, as scanning permission was neither requested of nor granted from the respective site owners.

IV. HYDRATION TRACKER

The hydration tracker consists of a physical water bottle designed for child use and an associated mobile application. The hydration tracker was released in 2016, and the mobile application has 5000+ installs from the Google Play store as of July 2018. An alternative version of the water bottle was released in 2018 with additional features, indicating continued sales of the toy. The user, likely a child, can create a profile on the application, and after entering biographical and personal information such as name, age, weight, and height, a water consumption goal is computed. Water consumed from the bottle is logged and can be viewed from within the application. The water bottle has a graphical interface for monitoring daily consumption goals. The user can also add a profile picture that appears inside the application. This photograph is likely of the child, although privacy-conscious users may choose a nonidentifying photograph.

A. Network Behavior

The water bottle is equipped with an 802.11 b/g/n WiFi card, but no Bluetooth functionality. The mobile application,

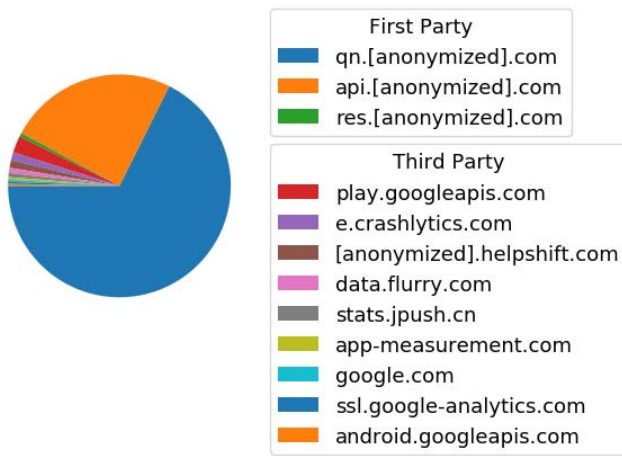


Fig. 2. Hosts communicating with the hydration tracker by percentage of bytes transferred over a 5-min traffic recording.

by virtue of running on a smartphone, can communicate via WiFi and Bluetooth.

Decompiling the mobile application source code revealed numerous imported libraries for communication with third-party analytics and performance monitoring services. These services include Yahoo's Flurry Analytics,¹ Google Analytics,² Crashlytics,³ and a Chinese analytics platform.⁴

Summary flow statistics reveal that the hydration tracker communicates with approximately 12 remote hosts. Fig. 2 plots these hosts weighted by how many bytes they exchanged with the hydration tracker. The hosts fall into two categories: 1) servers belonging to the manufacturer of the toy and 2) servers providing third-party analytics and performance monitoring. All of the HTTP connections to third-party platforms were encrypted over secure socket layer (SSL), and all connections to manufacturer-owned servers were unencrypted, vanilla HTTP.

B. Vulnerabilities

Analysis of the hydration tracker revealed several vulnerabilities, primarily due to lack of encryption and authentication of HTTP sessions.

1) *HTTP Communications With Cloud Servers*: All communications between the hydration tracker and manufacturer-owned servers occur via unencrypted and unauthenticated HTTP GET and POST requests. These communications include requests for static content, such as background images in the application, as well as for dynamic content related to user behaviors. When the user drinks from the water bottle, the bottle makes an HTTP POST request to report the event. An attacker could easily observe the contents of these requests, learn when a user is interacting with the device, and/or spoof a response with arbitrary content [Fig. 3(a)]. This content could contain executable data that triggers remote code execution in the application or smart water bottle.

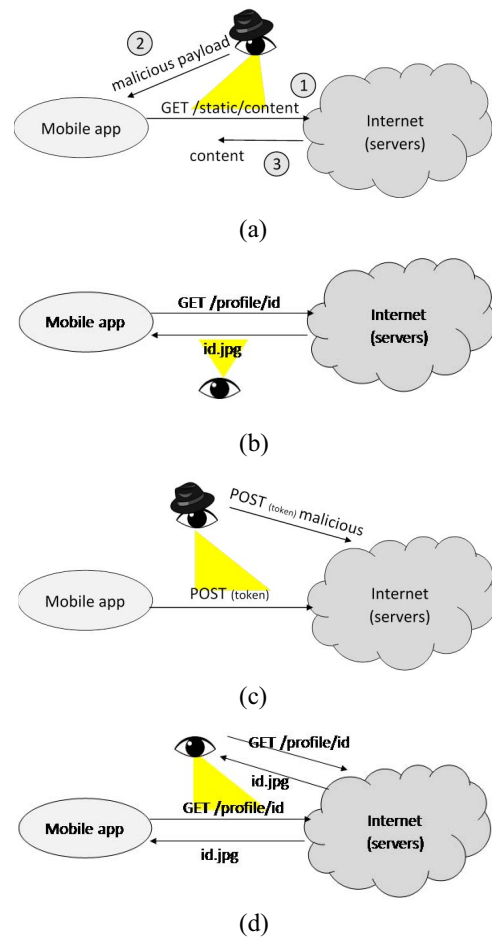


Fig. 3. Malicious actions enabled by discovered security vulnerabilities. (a) Malicious actor observing an unencrypted GET request and spoofing the response with a malicious payload. (b) Malicious actor observing PII in an unencrypted server response. (c) Malicious actor obtaining a valid token from a POST request and spoofing a new request to the server with a malicious payload. (d) Malicious actor observing a GET request and spoofing it to the server to receive a response.

Unencrypted HTTP GET requests are also used to fetch the user's profile picture upon loading or restarting the mobile application. An adversary could eavesdrop on these communications and observe the profile photographs of children using the hydration tracker, a clear privacy breach [Fig. 3(b)].

2) *POST Token Reuse*: In addition to being unencrypted, POST requests from the hydration tracker reporting drinking events reuse an authentication token transmitted as an HTTP header (this appears to be a custom header as it is not specified in the relevant HTTP RFCs). An attacker could observe the token of a legitimate user by eavesdropping a valid request and then spoof valid requests to the server with arbitrary content [Fig. 3(c)]. This could allow remote code execution on the server and be used as an attack vector to dump the server's database filled with sensitive user data.

Securing POST requests with HTTPS would not prevent request spoofing. If an attacker was able to guess a valid token, they could establish their own secure session to the server and continue spoofing requests pretending to be a legitimate user. The tokens used for these POST requests should have

¹[Online]. Available: <https://developer.yahoo.com/analytics/>

²[Online]. Available: <https://analytics.google.com/>

³[Online]. Available: <http://try.crashlytics.com>

⁴[Online]. Available: <https://www.jiguang.cn>

a time-to-live and be periodically refreshed to minimize the window of opportunity an attacker has to exploit a valid token.

3) *No Authentication for User Profile Photographs*: HTTP GET requests for user profile pictures are not only unencrypted but also unauthenticated. An attacker able to generate valid requests could receive the profile pictures of any hydration tracker user anywhere in the world, even if the attacker is not able to directly eavesdrop hydration tracker communications.

Even if the hydration tracker were updated to use HTTPS, this vulnerability would not be solved. An attacker could just make an HTTPS connection to the server and spoof the GET request over an encrypted connection [Fig. 3(d)]. An authentication mechanism is necessary to verify that user making the request has access rights to the profile picture.

4) *Continued Availability of Overwritten Profile Photographs*: After a user overwrites their current profile picture with a new one, their old profile picture is still accessible via the original HTTP GET request. This is functionally unnecessary, as there is currently no feature of the application to switch to a previous profile photograph. Moreover, this continued availability makes the window of opportunity for an attacker to obtain a profile photograph via sniffing or spoofing essentially infinite, in addition to needlessly polluting the space of valid photograph tokens and thus increasing the chances of a random guess corresponding to a valid token.

5) *Asymmetric HTTP Response Codes Allow Profile Photograph ID Discovery*: HTTP GET requests for user profile picture contain a unique 12-character identification token. However, requests with a truncated 3-character token receive differentiated responses depending on whether the three characters are a prefix of a legitimate token. Specifically, prefixes of real user tokens yield HTTP 301 Moved Permanently response codes, while prefixes of invalid tokens yield HTTP 404 Not Found response codes. Combined with the previously described lack of authentication for profile photograph GET requests, this implementation flaw substantially reduces the time required to guess valid picture tokens (Section IV-C).

6) *Personally Identifiable Information in Crash Reports*: During a transient network error, the mobile application sends a crash report to a third-party analytics platform containing PII, including the name, gender, birthday, and weight of the user (Fig. 4). Although this information was sent encrypted over HTTPS, it is unlikely that a third-party analytics platform needs this sensitive data. The application should exclude these data from crash reports.

C. Profile Picture Mining

This section outlines an attack against the hydration tracker to obtain profile pictures of arbitrary users without access to the device.

When the hydration tracker mobile application launches, and periodically thereafter, it makes an unencrypted HTTP GET request containing a unique 12-character identification token to a manufacturer-owned server (Fig. 5). If the token matches an existing user's picture, the server will respond with that picture.

```
created_date='null', nickname='Gordon1', weight='null', gender='boy',
birthday='2010-01-01 13:44:21', habitScore=null, portraitX1Url='null',
portraitX2Url='null', portraitX3Url='null', latitude=null, longitude=null,
portrait_update_time='null', habitScore_update_time='null', has_pet=false,
has_cup=false, update=true,
portraitPath='/data/data/com. [anonymized] /temp/temp1511721859833.png',
portraitPathX2='null', portraitPathX3='null', RecommendDrinkNum=null,
drinkNum=null, newFriendLabelTime='null', isNewFriend=false, unit=lbs,
weight_lbs=25,
```

Fig. 4. Excerpt of crash report containing sensitive information sent to a third-party analytics service from the hydration tracker toy.

```
GET /resource/profile/FZH/FZH930NBDL6Y.160.jpg'
```

prefix
suffix
prefix
(again)

Fig. 5. Example HTTP GET request for hydration tracker user profile picture with 3-character prefix labeled.

Because the request for the picture is unauthenticated, an attacker can simply send requests with tokens for pictures from arbitrary users. Although the attacker does not know which picture tokens correspond to existing photographs, they can repeatedly attempt to guess valid tokens until the server returns a picture.

A valid token consists of 12 characters, drawn from the uppercase alphanumeric alphabet. The uppercase alphanumeric alphabet has 36 entries, namely the 10 digits and the 26 uppercase letters A–Z. The number of valid tokens that can be created from this alphabet is 36^{12} .

It is possible to discover whether 3-character strings are prefixes of existing tokens separately from discovering entire tokens. The first three characters of a token are included twice in a GET request URL, first as a separate prefix string and then in the entire token (Fig. 5). An HTTP 301 response code indicates that a guessed prefix is valid, while an HTTP 404 indicates that the prefix is invalid. This ability to discover token prefixes considerably reduces the expected runtime to guess and verify tokens of existing user profile pictures.

An attacker would need to test 36^3 prefixes to discover all p valid prefixes. This would take less than 2 h for an HTTP response rate of 200 ms. The attacker would then need to test $p \times 36^9$ suffixes to obtain the profile photographs of all users of the hydration tracker. This would take more than 10^5 years with the same response rate—far too long for a reasonable attack. However, the attacker may be satisfied with finding only a fraction of all the profile pictures. In this case, the expected runtime of the algorithm scales down proportionally to the desired fraction.

Furthermore, an attacker could also parallelize the attack by testing guessed tokens on many machines. Care would have to be taken in seeding the pseudorandom number generators of different instances of the algorithm to minimize overlap during the permutation of the tokenspace.

In spite of the combinatoric infeasibility of testing all possible picture tokens, the fact that any user profile photographs could be recovered by an attacker raises serious privacy concerns.

D. Violations of COPPA and Privacy Policy

The hydration tracker violates COPPA in two ways. First and foremost, the profile picture mining attack and its enabling vulnerabilities allow unaffiliated actors to obtain profile pictures of arbitrary users without access to the device. Since this device is targeted toward children, and user profile photographs are likely of children, this vulnerability is in clear violation of COPPA section 312.8, namely that the manufacturer must have “reasonable procedures to protect the confidentiality, security, and integrity of personal information collected from children” and “take reasonable steps to release children’s personal information only to service providers and third parties who are capable of maintaining the confidentiality, security and integrity of such information” [2].

The hydration tracker also violates COPPA’s data retention policy. After changing the photograph associated with a given profile to a new picture, the URL used to retrieve the old photograph still works. This violates COPPA section 312.10, which mandates that the manufacturer “shall retain personal information collected online from a child for only as long as is reasonably necessary to fulfill the purpose for which the information was collected” and that the manufacturer “must delete such information using reasonable measures to protect against unauthorized access to, or use of, the information in connection with its deletion” [2].

The hydration tracker also violates its own privacy policy with the vulnerabilities in Section IV-B. The privacy policy states that users’ personal information is “contained behind secured networks ... accessible by a limited number of persons who have special access rights” and is “encrypted via Secure Socket Layer (SSL) technology.” These assertions are clearly false.

V. SMART PET

The smart pet is a nonconnected plush toy with a companion mobile application, available on both iOS and Android platforms. Upon creating an account through the user interface, a virtual “pet” verbally prompts the user for their name and birthday. Afterward, the user inserts the phone into the plush toy and plays with it in interactive skits. The smart pet was released in 2013, and the mobile application has 10000+ installs from the Google Play store as of July 2018. The company which originally made the smart pet has since partnered with a major global toy manufacturer (top five worldwide by revenue) and continues active development of new smart pet versions.

A. Network Behavior

Because the smart pet system is a standalone mobile application, it can speak the same network protocols as the smartphone. On the Android platform, the application requests permissions to access and change WiFi connectivity state. The description of the application in the Google Play store states that Bluetooth capability is necessary for certain features. Decompiling the binary revealed two imported libraries for third party analytics platforms: 1) Google Analytics and

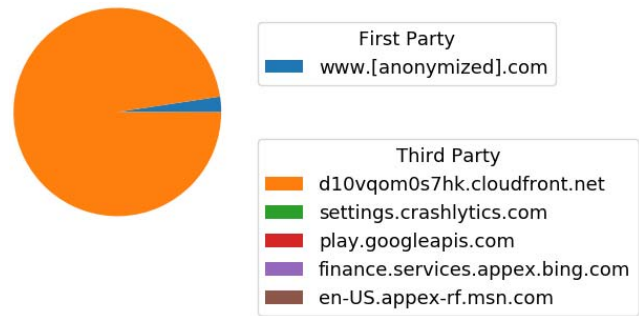


Fig. 6. Hosts communicating with smart pet by percentage of bytes transferred over a 4-min traffic recording.

```
String NOOK_ALLPACK_SERVICE_ID = "64bbeb57b650d9864a38a1f31e5f4f8b";
String NOOK_ALLPACK_SERVICE_INAPP_SECRET = "9b610aec51819425519a63d8a0eb2861";
String NOOK_PACK_SERVICE_ID = "64bbeb57b650d9864a38a1f31e5f4f8b";
String NOOK_PACK_SERVICE_INAPP_SECRET = "9b610aec51819425519a63d8a0eb2861";
```

Fig. 7. Secret constants stored in plaintext in the smart pet source code.

2) Crashlytics. Summary conversation and endpoint statistics reveal that the smart pet communicates with approximately six remote hosts (Fig. 6). The hosts are composed of manufacturer-owned servers, third-party analytics platforms, Microsoft news services, and a large content distribution network. Sessions to Microsoft news domains (msn.com) are over unencrypted HTTP. Sessions to all other hosts are encrypted over SSL.

B. Vulnerabilities

The smart pet analysis revealed vulnerabilities involving constant storage, encryption, and authentication.

1) *Constants File Exposes Sensitive Tokens*: The source code of the smart pet contains a Java file with several constants in cleartext (Fig. 7). Most of them are innocuous; however, two constants in particular appear to be secret tokens for the Nook in-app purchase API.⁵

- 1) NOOK_ALLPACK_SERVICE_INAPP_SECRET.
- 2) NOOK_PACK_SERVICE_INAPP_SECRET.

Rather than storing these tokens in plaintext in source code, these secret tokens should be stored in an encrypted format and only be decrypted in memory. Otherwise, an attacker could make API calls for in-app purchases on behalf of the client.

2) *Cleartext HTTP for XML Files*: The mobile application makes unencrypted HTTP GET requests for XML files containing news headlines from several Microsoft domains. An attacker could intercept, read, and spoof these requests or responses with little effort. In the worst case, depending on how the XML is parsed and used, this could lead to remote code execution. Maliciously formatted XML has been known to cause remote code execution flaws in server software such as Apache Struts, a vulnerability which was exploited in the 2017 Equifax breach [27]. The GET requests for these XML files should be made over HTTPS to mitigate response-spoofing attacks and snooping by malicious agents.

⁵The Nook API is deprecated as of March 2016, however, this toy was available at least as early as August 2014, meaning this vulnerability was live for nearly two years.

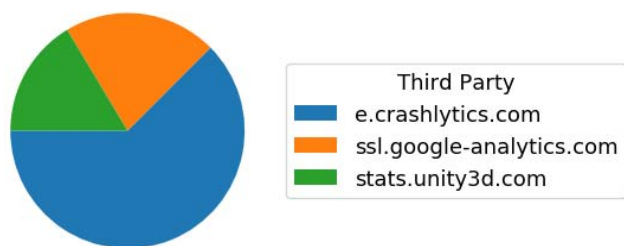


Fig. 8. Hosts communicating with fitness band by percentage of bytes transferred over a 4-min traffic recording. No communications with first-party servers were observed.

VI. FITNESS BAND

The fitness band consists of a wristband and an associated mobile application. After synchronizing the wristband to the mobile application, the user can play games inside the application. Characters in the game are controlled by movement of the arm wearing the wristband. The fitness band was released in 2014, and the mobile application has 1000+ installs from the Google Play store as of July 2018. Active development of the band seems to have stopped. The last update to the mobile application occurred in 2016, but the physical band is still available for purchase on Amazon.com as of July 2018.

A. Network Behavior

The wristband is Bluetooth enabled but has no WiFi capability. The application, by virtue of running on a smartphone, can communicate via WiFi and Bluetooth.

Decompiling the binary to source code revealed four imported third-party libraries for analytics and performance monitoring. These include Yahoo's Flurry Analytics, Google Analytics, Crashlytics, and Unity 3D statistics.⁶

Summary conversation and endpoint statistics reveal that the fitness band communicates significantly with approximately three remote hosts (Fig. 8). The hosts are only third-party analytics platforms. All connections are encrypted using SSL.

B. Vulnerabilities

No vulnerabilities were discovered in the fitness band's decompiled source code or its small amount of communication with remote hosts.

VII. DISCUSSION

This section reflects on the current state of smart toy security based on the results of this paper.

A. Privacy Policy Violations

Smart toy manufacturers promise strong privacy and security guarantees explicitly in their own privacy policies and implicitly by adhering to COPPA, but fall short during implementation. For example, the hydration tracker's privacy policy claims that PII is protected "behind secured networks... accessible by a limited number of persons who have special access rights" and "encrypted via Secure Socket

Layer (SSL) technology." As shown in Section IV, this is clearly not the case. Furthermore, this pattern of good privacy posture being let down by weak implementation is not unique to the toys in this paper. Recently, researchers have shown that in spite of an encryption mechanism intended to preserve the privacy of children's voice recordings in transit, a particular toy's usage of a fixed set of keys made it possible to completely decrypt any voice communication between the toy and its server [8].

Such violations invite legal action against the toy manufacturers. In January 2018, the FTC settled charges against toy manufacturer VTech for "failing to take reasonable steps to secure the [children's] data it collected" and other COPPA violations [28]. In addition to federal action, state attorneys general have settled with technology companies for COPPA and privacy policy violations [29]. Our results should encourage researchers to continue performing security analyses of Internet-connected products and to provide information about discovered vulnerabilities to the FTC or state attorneys general.

B. Overlapping Third-Party Analytics

Each smart toy talks to a similar set of third-party analytics and performance monitoring platforms. Google Analytics and Crashlytics libraries were included in the source code of all three toys. Yahoo's Flurry Analytics was included in the source code of two of the toys. This suggests that a small set of platforms have high visibility into a broad set of smart toys. Coupled with over-reporting of PII to analytics services (Section IV-B6), these platforms could be receiving and storing more sensitive data than users expect.

This privacy vulnerability is similar to well-studied concerns about third-party Web tracking. The incorporation of third-party content, such as advertisements or social media integration, on numerous websites allows third-party companies to track individuals' browsing behavior across large portions of the Web [30]. If third-party analytics companies receive data from numerous smart toys, they could similarly construct detailed profiles of individual children's behaviors, as well as large-scale datasets about smart toy users of value to manufacturers and distributors. Further research should investigate the proliferation of these analytics platforms across a wider set of smart toys, as well as the extent of user information they collect.

C. Lack of Encryption With First-Party Servers

We found several network security vulnerabilities in communications between smart toy systems and first-party manufacturer-owned servers, most due to lack of encryption or authentication. In comparison, communications between smart toy systems and third-party analytics servers were all secure. This is likely because third-party analytics libraries tend to be from mature companies that use encryption and authentication best practices, while first-party communications are implemented by toy manufacturers without the time, resources, or know-how to use encrypted protocols. This is a phenomenon seen across the IoT space, as manufacturers of physical objects

⁶[Online]. Available: <https://hwstats.unity3d.com/>

with little software development experience begin creating “smart” products.

This indicates an analogy to the early days of the Web, where encryption and other security best practices were much less well-known and widely adopted. Web security has gradually improved due to a variety of factors, including site hosting services with security baked-in, improved Web development libraries reducing the potential for programming errors and simplifying SSL/TLS support, improved browser-based security features, new versions of Web standards, and more. This comparison invites IoT researchers and manufacturers to look to the systems that have improved Web security as inspiration for simplifying and securing IoT development.

This observation also supports the role that regulation can play in securing smart toys and other IoT devices. The European Union’s General Data Privacy Regulation encourages, but does not mandate, the use of encryption to protect infringement (Recital 83) [31]. Industry advisory groups have made similar recommendations for IoT device design and development, including using “strong authentication by default” and following “security and cryptography best practices” [32]. Mandating SSL/TLS or such best practices would prevent vendors with little security background from releasing devices with egregious cleartext vulnerabilities. The availability of SSL/TLS libraries in every commonly used language for IoT development would already prevent such regulations from being overly burdensome to manufacturers. Excessive manufacturer effort to implement security features can no longer be an excuse for insecure Internet-connected toys that place children at risk.

VIII. FUTURE DIRECTIONS

While this paper discovered several security vulnerabilities and privacy policy violations, additional research is needed to protect children from insecure Internet-connected toys.

A. Analyzing Additional Toys

The three toys we analyzed demonstrate the breadth of network security vulnerabilities present in Internet-connected children’s products and detectable using standard analysis techniques. While these case studies provide further evidence that IoT manufacturers need to place higher priority on security and indicate specific problem areas for attention, a comprehensive view of smart toy security would require analyzing many additional toys and repeating these analyses as the toys and corresponding mobile applications receive software updates.

Since this project was focused on the three toys requested by the state attorney general, we leave repeated analyses of additional toys for future work. The development of an automated auditing tool, as discussed in the following section, would also greatly simplify the process of analyzing many toys in parallel.

B. Automated Toy Auditing

The static and dynamic analyses we performed in this paper mostly involved manual or custom-programmed data

inspection, whether the data was decompiled source code, privacy policies, or network traffic traces. Researchers have long proposed and developed automated network security analysis tools, both for IoT devices [33] and more traditional network configurations [34], [35]. There are also many commercially available analysis tools which perform online and offline security audits of network traffic or source code. However, none of these tools connect network or code analysis with product privacy policies. The development of an automated analysis tool for identifying device privacy and security vulnerabilities and determining whether they constitute privacy policy breaches would greatly reduce regulator burden, especially in the IoT market where new devices and device updates are released faster than they can be manually audited. However, creating such a tool would be nontrivial, because privacy policies are often written with intentionally vague wording that does not directly map to concrete device behaviors [36]. A successful automated tool would need to map privacy policy text to verifiable security or privacy properties and check devices for these properties.

IX. CONCLUSION

This paper has examined three commercially available IoT toys to gain a deeper understanding of the smart toy security and privacy landscape. Through a combination of static and dynamic analysis, several previously undisclosed vulnerabilities were discovered, including neglected encryption and authentication, POST token reuse, sensitive user information in crash reports, and secret keys in source code. These vulnerabilities violate the toys’ individual privacy policies as well as federal COPPA regulations for handling children’s data. Additionally, a small set of third-party analytics platforms receives data from all examined toys, possibly allowing detailed user data collection similar to third-party Web tracking. These results indicate that Internet-connected children’s toys require continued security and privacy auditing, and that further work is needed to help IoT toy manufacturers improve security and privacy development practices.

REFERENCES

- [1] S. Gibbs. (Nov. 2015). *Hackers Can Hijack Wi-Fi Hello Barbie to Spy on Your Children*. Accessed: Nov. 15, 2016. [Online]. Available: <https://www.theguardian.com/technology/2015/nov/26/hackers-can-hijack-wi-fi-hello-barbie-to-spy-on-your-children>
- [2] Federal Trade Commission. (1998). *Children’s Online Privacy Protection Rule (‘COPPA’)*. [Online]. Available: <https://www.ftc.gov/enforcement/rules/rulemaking-regulatory-reform-proceedings/childrens-online-privacy-protection-rule>
- [3] K. Munro. (Jan. 2015). *Making Children’s Toys Swear*. Accessed: Jan. 23, 2015. [Online]. Available: <https://www.pentestpartners.com/security-blog/making-childrens-toys-swear/>
- [4] D. Cooper. (2017). *Researchers Find Another Smart Toy That’s Easy to Hack*. [Online]. Available: <https://www.engadget.com/2017/12/08/teksta-toucan-can-listen-to-kids-researchers-security/>
- [5] L. Franceschi-Bicchieri. (2017). *Internet of Things Teddy Bear Leaked 2 Million Parent and Kids Message Recordings*. [Online]. Available: https://motherboard.vice.com/en_us/article/pgwean/internet-of-things-teddy-bear-leaked-2-million-parent-and-kids-message-recordings
- [6] E. Leamy, *The Danger of Giving Your Child ‘Smart Toys’*, Washington Post, Washington, DC, USA, 2017.
- [7] B. Barrett. (2017). *Don’t Get Your Kid an Internet-Connected Toy*. [Online]. Available: <https://www.wired.com/story/dont-gift-internet-connected-toys/>

- [8] J. Valente and A. A. Cardenas, "Security & privacy in smart toys," in *Proc. Workshop Internet Things Security Privacy*, Dallas, TX, USA, 2017, pp. 19–24.
- [9] M. L. Jones and K. Meurer, "Can (and should) hello Barbie keep a secret?" in *Proc. IEEE Int. Symp. Ethics Eng. Sci. Technol. (ETHICS)*, Vancouver, BC, Canada, 2016, pp. 1–6.
- [10] J.-S. Pleban, R. Band, and R. Creutzburg, "Hacking and securing the AR.Drone 2.0 quadcopter: Investigations for improving the security of a toy," in *Proc. Mobile Devices Multimedia Enabling Technol. Algorithms Appl.*, vol. 9030, Feb. 2014, Art. no. 90300L.
- [11] E. McReynolds *et al.*, "Toys that listen: A study of parents, children, and Internet-connected toys," in *Proc. CHI Conf. Human Factors Comput. Syst.*, Denver, CO, USA, 2017, pp. 5197–5207.
- [12] A. Manches, P. Duncan, L. Plowman, and S. Sabeti, "Three questions about the Internet of Things and children," *TechTrends*, vol. 59, no. 1, pp. 76–83, 2015.
- [13] J. Haynes, M. Ramirez, T. Hayajneh, and M. Z. A. Bhuiyan, "A framework for preventing the exploitation of IoT smart toys for reconnaissance and exfiltration," in *Proc. Int. Conf. Security Privacy Anonymity Comput. Commun. Stor.*, Guangzhou, China, 2017, pp. 581–592.
- [14] P. C. K. Hung, F. Iqbal, S.-C. Huang, M. Melaisi, and K. Pang, "A glance of child's play privacy in smart toys," in *Proc. Int. Conf. Cloud Comput. Security*, Nanjing, China, 2016, pp. 217–231.
- [15] B. Yankson, F. Iqbal, and P. C. K. Hung, "Privacy preservation framework for smart connected toys," in *Computing in Smart Toys*. Cham, Switzerland: Springer, 2017, pp. 149–164.
- [16] L. Rafferty *et al.*, "Towards a privacy rule conceptual model for smart toys," in *Computing in Smart Toys*. Cham, Switzerland: Springer, 2017, pp. 85–102.
- [17] S. Yong, D. Lindskog, R. Ruhl, and P. Zavorsky, "Risk mitigation strategies for mobile Wi-Fi robot toys from online pedophiles," in *Proc. IEEE 3rd Int. Conf. Privacy Security Risk Trust Comput.*, Boston, MA, USA, Oct. 2011, pp. 1220–1223.
- [18] I. Reyes *et al.*, "'Won't somebody think of the children?' Examining COPPA compliance at scale," *Proc. Privacy Enhanc. Technol.*, vol. 2018, no. 3, pp. 63–83, 2018.
- [19] D. Zhu, J. Jung, D. Song, T. Kohno, and D. Wetherall, "Tainteraser: Protecting sensitive data leaks using application-level taint tracking," *ACM SIGOPS Oper. Syst. Rev.*, vol. 45, no. 1, pp. 142–154, 2011.
- [20] (2018). *Jadx Dex to Java Compiler*. [Online]. Available: <https://github.com/skylot/jadx>
- [21] N. Aphorpe. *IoT-Inspector*. [Online]. Available: <https://github.com/NoahAphorpe/iot-inspector>
- [22] A. Cortesi. *Mitmproxy*. [Online]. Available: <https://mitmproxy.org>
- [23] *Tshark*. [Online]. Available: <https://www.wireshark.org/docs/man-pages/tshark.html>
- [24] *Wireshark*. [Online]. Available: <https://www.wireshark.org>
- [25] P. Biondi, G. Valadon, and P. Lalet. *Scapy*. [Online]. Available: <http://www.secdev.org/projects/scapy/>
- [26] G. L. Fyodor. *Nmap*. [Online]. Available: <https://nmap.org>
- [27] NIST. (Sep. 2017). *CVE-2017-5638*. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2017-5638>
- [28] Federal Trade Commission. (2018). *Electronic Toy Maker VTech Settles FTC Allegations That It Violated Children's Privacy Law and the FTC Act*. [Online]. Available: <https://www.ftc.gov/news-events/press-releases/2018/01/electronic-toy-maker-vtech-settles-ftc-allegations-it-violated>
- [29] New York State Attorney General's Press Office. (2017). *A.G. Schneiderman Announces \$100,000 Settlement With TRUSTe Over Flawed Privacy Certification Program for Popular Children's Websites*. [Online]. Available: <https://ag.ny.gov/press-release/ag-schneiderman-announces-100000-settlement-truste-over-flawed-privacy-certification>
- [30] J. R. Mayer and J. C. Mitchell, "Third-party Web tracking: Policy and technology," in *Proc. IEEE Symp. Security Privacy*, San Francisco, CA, USA, 2012, pp. 413–427.
- [31] G. Blogger and J. Walker. (Apr. 2018). *What GDPR Means for Encryption*. [Online]. Available: <https://www.venafi.com/blog/what-gdpr-means-encryption>
- [32] "Internet of Things (IoT) security and privacy recommendations," Broadband Internet Technical Advisory Group, Denver, CO, USA, Rep., 2016. [Online]. Available: [https://www.bitag.org/documents/BITAG_Report_-_Internet_of_Things_\(IoT\)_Security_and_Privacy_Recommendations.pdf](https://www.bitag.org/documents/BITAG_Report_-_Internet_of_Things_(IoT)_Security_and_Privacy_Recommendations.pdf)
- [33] A. K. Simpson, F. Roesner, and T. Kohno, "Securing vulnerable home IoT devices with an in-hub security manager," in *Proc. IEEE Int. Conf. Pervasive Comput. Commun. Workshops (PerCom Workshops)*, Kailua, HI, USA, 2017, pp. 551–556.
- [34] M. Roesch, "Snort—Lightweight intrusion detection for networks," in *Proc. Lisa*, Seattle, WA, USA, 1999, pp. 229–238.
- [35] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: A logic-based network security analyzer," in *Proc. USENIX Security Symp.*, vol. 14. Baltimore, MD, USA, 2005, p. 8.
- [36] J. Bhatia, T. D. Breaux, J. R. Reidenberg, and T. B. Norton, "A theory of vagueness and privacy risk perception," in *Proc. IEEE 24th Int. Proc. Requirements Eng. Conf. (RE)*, Beijing, China, 2016, pp. 26–35.

Authors' photographs and biographies not available at the time of publication.